

The Gauss-Seidel Method

A Complete Lecture with Mathematical Framework and Applications

Zamir Hussain
Department of Mathematics
University of Wah

February 4, 2026

Week 8: Numerical Linear Algebra

This lecture covers Week 8 material on Iterative Methods in Numerical Linear Algebra. The Gauss-Seidel method is a cornerstone technique for solving large sparse systems efficiently.

Learning Objectives

By the end of this lecture, you will be able to:

- Derive the Gauss-Seidel iterative formula from matrix decomposition
- Analyze convergence using eigenvalue and norm criteria
- Distinguish between well-conditioned and ill-conditioned systems
- Implement the algorithm step-by-step for 2×2 and 3×3 systems
- Compare Gauss-Seidel with Jacobi method convergence rates
- Calculate and interpret the Frobenius norm for iterative matrices

Theoretical Foundations

Core Mathematical Concepts:

- Matrix Decomposition: $A = I + L + U$
- Iterative Matrix: $C = -(I + L)^{-1}U$
- Spectral Radius: $\rho(C) = \max |\lambda_i|$
- Matrix Norms: Frobenius, Row-sum, Column-sum
- Condition Number: $\kappa(A) = \|A\| \|A^{-1}\|$
- Diagonal Dominance: $|a_{ii}| > \sum_{j \neq i} |a_{ij}|$

Real-World Applications

Applications in Engineering & Data Science:

- Circuit analysis in electrical engineering
- Finite element methods for structural analysis
- Google's PageRank algorithm (modified)
- Solving partial differential equations (PDEs)
- Image reconstruction in medical imaging
- Portfolio optimization in finance

Learning Resources

Recommended Resources:

- **Textbook:** Numerical Linear Algebra by Trefethen & Bau
- **Software:** MATLAB, Python (NumPy/SciPy), Julia
- **Online:** MIT OpenCourseWare 18.335, Numerical Recipes
- **Visualization:** GeoGebra for matrix norm visualization

Textbook Reference

Chapter Reference: Chapter 11 - Iterative Methods for Linear Systems

Advanced Topics: Successive Over-Relaxation (SOR), Krylov Subspace Methods, Preconditioning Techniques

1 Introduction to Iterative Methods

1.1 Why Iterative Methods?

For large systems of linear equations (e.g., 1000×1000), direct methods like Gaussian elimination become computationally expensive with complexity $O(n^3)$. Iterative methods provide an alternative approach that can be more efficient for **sparse systems** where most entries are zero.

Method Type	Complexity	Best For
Direct (Gaussian Elimination)	$O(n^3)$	Small, dense systems
Iterative (Gauss-Seidel)	$O(kn^2)$	Large, sparse systems

Table 1: Comparison of solution methods for $Ax = b$

1.2 The Gauss-Seidel Method

The Gauss-Seidel method solves:

$$Ax = b$$

where:

- A is $n \times n$ coefficient matrix
- x is unknown vector
- b is constant vector

Key Insight: Use the most recently computed values in each iteration.

2 Mathematical Derivation & Algorithm

2.1 Iterative Formula Derivation

Given: $\sum_{j=1}^n a_{ij}x_j = b_i$, for $i = 1, 2, \dots, n$

$$\begin{aligned}a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n &= b_1 \\a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n &= b_2 \\&\vdots \\a_{n1}x_1 + a_{n2}x_2 + \dots + a_{nn}x_n &= b_n\end{aligned}$$

Solve for x_i from the i -th equation:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij}x_j^{(k)} \right)$$

Note: k = iteration index, $x_j^{(k+1)}$ are newly computed values used immediately.

2.2 Matrix Formulation

Let $A = D + L + U$, where:

- D = diagonal matrix of A ($\text{diag}(a_{11}, a_{22}, \dots, a_{nn})$)
- L = strictly lower triangular part of A ($l_{ij} = a_{ij}$ if $i > j$, else 0)
- U = strictly upper triangular part of A ($u_{ij} = a_{ij}$ if $i < j$, else 0)

Step-by-Step Problem

Problem 1: Matrix Decomposition Given: $A = \begin{bmatrix} 4 & 1 & -1 \\ 2 & 6 & 0 \\ 1 & 2 & 5 \end{bmatrix}$
Decompose into D , L , and U components.

✓ Solution

$$D = \begin{bmatrix} 4 & 0 & 0 \\ 0 & 6 & 0 \\ 0 & 0 & 5 \end{bmatrix}, \quad L = \begin{bmatrix} 0 & 0 & 0 \\ 2 & 0 & 0 \\ 1 & 2 & 0 \end{bmatrix}, \quad U = \begin{bmatrix} 0 & 1 & -1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

2.3 Iterative Matrix Development

From $A\mathbf{x} = (D + L + U)\mathbf{x} = \mathbf{b}$:

Gauss-Seidel iteration: $(D + L)\mathbf{x}^{(k+1)} = \mathbf{b} - U\mathbf{x}^{(k)}$

$$\mathbf{x}^{(k+1)} = (D + L)^{-1}\mathbf{b} - (D + L)^{-1}U\mathbf{x}^{(k)}$$

2.3.1 Normalized Form (Unit Diagonal)

Scale so $a_{ii} = 1$: Multiply by D^{-1} :

$$D^{-1}A = I + D^{-1}L + D^{-1}U = I + \tilde{L} + \tilde{U}$$

Then:

$$\mathbf{x}^{(k+1)} = (I + \tilde{L})^{-1}D^{-1}\mathbf{b} - (I + \tilde{L})^{-1}\tilde{U}\mathbf{x}^{(k)}$$

The **iterative matrix** is:

$$C = -(I + \tilde{L})^{-1}\tilde{U}$$

3 Convergence Criteria & Analysis

3.1 Necessary and Sufficient Condition

[Convergence Theorem] The Gauss-Seidel method converges for any initial guess $\mathbf{x}^{(0)}$ **if and only if**:

$$\rho(C) < 1$$

where $\rho(C) = \max_i |\lambda_i(C)|$ is the **spectral radius** (maximum absolute eigenvalue).

3.2 Sufficient Conditions

1. **Strict Diagonal Dominance:**

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|, \quad \forall i = 1, \dots, n$$

2. **Symmetric Positive Definite:** If A is SPD, Gauss-Seidel converges.

3. **Matrix Norm Condition:** If $\|C\| < 1$ for any matrix norm, convergence is guaranteed.

4 Step-by-Step Problem Solving

Step-by-Step Problem

Problem 2: Complete Gauss-Seidel Solution Solve using Gauss-Seidel (3 iterations):

$$\begin{cases} 8x_1 + x_2 - x_3 = 8 \\ 2x_1 + x_2 + 9x_3 = 12 \\ x_1 - 7x_2 + 2x_3 = -4 \end{cases}$$

Start with $\mathbf{x}^{(0)} = [0, 0, 0]^T$.

✓ **Solution**

Step 1: Rearrange equations:

$$\begin{aligned}x_1 &= \frac{1}{8}(8 - x_2 + x_3) \\x_2 &= \frac{1}{1}(12 - 2x_1 - 9x_3) \\x_3 &= \frac{1}{2}(-4 - x_1 + 7x_2)\end{aligned}$$

Step 2: First iteration ($k = 0 \rightarrow k = 1$):

$$\begin{aligned}x_1^{(1)} &= \frac{1}{8}(8 - 0 + 0) = 1.0 \\x_2^{(1)} &= 12 - 2(1.0) - 9(0) = 10.0 \\x_3^{(1)} &= \frac{1}{2}(-4 - 1.0 + 7(10.0)) = \frac{1}{2}(65) = 32.5\end{aligned}$$

Step 3: Second iteration ($k = 1 \rightarrow k = 2$):

$$\begin{aligned}x_1^{(2)} &= \frac{1}{8}(8 - 10.0 + 32.5) = \frac{30.5}{8} = 3.8125 \\x_2^{(2)} &= 12 - 2(3.8125) - 9(32.5) = 12 - 7.625 - 292.5 = -288.125 \\x_3^{(2)} &= \frac{1}{2}(-4 - 3.8125 + 7(-288.125)) = \frac{1}{2}(-2024.6875) = -1012.34375\end{aligned}$$

Step 4: Third iteration ($k = 2 \rightarrow k = 3$):

$$\begin{aligned}x_1^{(3)} &= \frac{1}{8}(8 - (-288.125) + (-1012.34375)) = \frac{-716.21875}{8} = -89.5273 \\x_2^{(3)} &= 12 - 2(-89.5273) - 9(-1012.34375) = 12 + 179.0546 + 9111.09375 = 9302.14835 \\x_3^{(3)} &= \frac{1}{2}(-4 - (-89.5273) + 7(9302.14835)) = \frac{1}{2}(65200.56675) = 32600.2834\end{aligned}$$

Observation: Values are diverging! This suggests the system may not be diagonally dominant or ill-conditioned.

Check diagonal dominance:

- Row 1: $|8| > |1| + |-1| = 2$
- Row 2: $|1| > |2| + |9| = 11$
- Row 3: $|2| > |1| + |-7| = 8$

Not diagonally dominant - convergence not guaranteed!

5 Matrix Norm Analysis & Convergence Testing

Step-by-Step Problem

Problem 3: Iterative Matrix and Frobenius Norm For the normalized matrix:

$$A = \begin{bmatrix} 1 & 0.2 & -0.1 \\ 0.3 & 1 & 0.4 \\ -0.2 & 0.1 & 1 \end{bmatrix}$$

Calculate the iterative matrix C and its Frobenius norm.

✓ Solution

Step 1: Decompose $A = I + L + U$:

$$I = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad L = \begin{bmatrix} 0 & 0 & 0 \\ 0.3 & 0 & 0 \\ -0.2 & 0.1 & 0 \end{bmatrix}, \quad U = \begin{bmatrix} 0 & 0.2 & -0.1 \\ 0 & 0 & 0.4 \\ 0 & 0 & 0 \end{bmatrix}$$

Step 2: Compute $(I + L)^{-1}$:

$$I + L = \begin{bmatrix} 1 & 0 & 0 \\ 0.3 & 1 & 0 \\ -0.2 & 0.1 & 1 \end{bmatrix}$$

For lower triangular matrix, inverse is:

$$(I + L)^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ -0.3 & 1 & 0 \\ 0.23 & -0.1 & 1 \end{bmatrix}$$

Step 3: Compute $C = -(I + L)^{-1}U$:

$$\begin{aligned} C &= - \begin{bmatrix} 1 & 0 & 0 \\ -0.3 & 1 & 0 \\ 0.23 & -0.1 & 1 \end{bmatrix} \begin{bmatrix} 0 & 0.2 & -0.1 \\ 0 & 0 & 0.4 \\ 0 & 0 & 0 \end{bmatrix} \\ &= - \begin{bmatrix} 0 & 0.2 & -0.1 \\ 0 & -0.06 & 0.07 \\ 0 & 0.046 & -0.087 \end{bmatrix} \\ &= \begin{bmatrix} 0 & -0.2 & 0.1 \\ 0 & 0.06 & -0.07 \\ 0 & -0.046 & 0.087 \end{bmatrix} \end{aligned}$$

Step 4: Calculate Frobenius norm:

$$\begin{aligned} \|C\|_F &= \sqrt{\sum_{i=1}^3 \sum_{j=1}^3 c_{ij}^2} \\ &= \sqrt{(-0.2)^2 + (0.1)^2 + (0.06)^2 + (-0.07)^2 + (-0.046)^2 + (0.087)^2} \\ &= \sqrt{0.04 + 0.01 + 0.0036 + 0.0049 + 0.002116 + 0.007569} \\ &= \sqrt{0.068185} \approx 0.261 \end{aligned}$$

Interpretation: Since $\|C\|_F = 0.261 < 1$, convergence is guaranteed.

6 Ill-Conditioned Systems & Sensitivity Analysis

6.1 Condition Number Concept

The condition number measures sensitivity:

$$\kappa(A) = \|A\| \cdot \|A^{-1}\|$$

- $\kappa(A) \approx 1$: Well-conditioned

- $\kappa(A) \gg 1$: Ill-conditioned

Step-by-Step Problem

Problem 4: Ill-Conditioned 2×2 System Analyze the system:

$$\begin{bmatrix} 2 & 1 \\ 2 & 1.0001 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 3 \\ 3.0001 \end{bmatrix}$$

and then with perturbed RHS: $\begin{bmatrix} 3 \\ 3.0002 \end{bmatrix}$

✓ Solution

Original system solution:

$$\begin{bmatrix} 2 & 1 \\ 2 & 1.0001 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 3 \\ 3.0001 \end{bmatrix}$$

Solve: $2x_1 + x_2 = 3$ and $2x_1 + 1.0001x_2 = 3.0001$

Subtract equations: $0.0001x_2 = 0.0001 \Rightarrow x_2 = 1$

Then $2x_1 + 1 = 3 \Rightarrow x_1 = 1$

Solution: $\mathbf{x} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$

Perturbed system:

$$\begin{bmatrix} 2 & 1 \\ 2 & 1.0001 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 3 \\ 3.0002 \end{bmatrix}$$

$2x_1 + x_2 = 3$ and $2x_1 + 1.0001x_2 = 3.0002$

Subtract: $0.0001x_2 = 0.0002 \Rightarrow x_2 = 2$

Then $2x_1 + 2 = 3 \Rightarrow x_1 = 0.5$

New solution: $\mathbf{x} = \begin{bmatrix} 0.5 \\ 2 \end{bmatrix}$

Analysis:

- Change in RHS: 0.0001 (0.0033% change)
- Change in solution: x_1 : $1 \rightarrow 0.5$ (50% change!), x_2 : $1 \rightarrow 2$ (100% change!)
- **This is ill-conditioning!**

Calculate condition number:

$$\det(A) = 2(1.0001) - 2(1) = 0.0002$$

$$A^{-1} = \frac{1}{0.0002} \begin{bmatrix} 1.0001 & -1 \\ -2 & 2 \end{bmatrix} = \begin{bmatrix} 5000.5 & -5000 \\ -10000 & 10000 \end{bmatrix}$$

Using -norm: $\|A\|_{\infty} = \max(3, 3.0001) = 3.0001$

$\|A^{-1}\|_{\infty} = \max(10000.5, 20000) = 20000$

$$\kappa_{\infty}(A) = 3.0001 \times 20000 \approx 60002$$

Verdict: Extremely ill-conditioned!

7 Comparison: Gauss-Seidel vs. Jacobi Method

Aspect	Jacobi Method	Gauss-Seidel Method
Update Rule	Uses only previous iteration	Uses most recent values
Matrix Form	$\mathbf{x}^{(k+1)} = D^{-1}\mathbf{b} - D^{-1}(L + U)\mathbf{x}^{(k)}$	$\mathbf{x}^{(k+1)} = (D + L)^{-1}\mathbf{b} - (D + L)^{-1}U\mathbf{x}^{(k)}$
Convergence Rate	Generally slower	Generally faster (1.5-2×)
Storage	2 vectors needed	1 vector sufficient
Parallelization	Easily parallelizable	Sequential dependencies
Memory	Less efficient	More efficient

Table 2: Comparison of iterative methods

7.1 Numerical Comparison

Step-by-Step Problem

Problem 5: Jacobi vs. Gauss-Seidel Comparison For the system (diagonally dominant):

$$\begin{cases} 10x_1 + 2x_2 + x_3 = 13 \\ x_1 + 5x_2 + x_3 = 7 \\ 2x_1 + 3x_2 + 10x_3 = 15 \end{cases}$$

Compare first 2 iterations starting from $[0, 0, 0]^T$.

✓ Solution

Jacobi Method: Iteration 1:

$$x_1^{(1)} = (13 - 0 - 0)/10 = 1.3$$

$$x_2^{(1)} = (7 - 0 - 0)/5 = 1.4$$

$$x_3^{(1)} = (15 - 0 - 0)/10 = 1.5$$

Iteration 2:

$$x_1^{(2)} = (13 - 2(1.4) - 1.5)/10 = 0.87$$

$$x_2^{(2)} = (7 - 1.3 - 1.5)/5 = 0.84$$

$$x_3^{(2)} = (15 - 2(1.3) - 3(1.4))/10 = 0.76$$

Gauss-Seidel Method: Iteration 1:

$$x_1^{(1)} = (13 - 0 - 0)/10 = 1.3$$

$$x_2^{(1)} = (7 - 1.3 - 0)/5 = 1.14$$

$$x_3^{(1)} = (15 - 2(1.3) - 3(1.14))/10 = 0.898$$

Iteration 2:

$$x_1^{(2)} = (13 - 2(1.14) - 0.898)/10 = 1.0812$$

$$x_2^{(2)} = (7 - 1.0812 - 0.898)/5 = 1.00416$$

$$x_3^{(2)} = (15 - 2(1.0812) - 3(1.00416))/10 = 0.990352$$

Comparison:

- After 2 iterations, Gauss-Seidel is closer to exact solution $[1, 1, 1]^T$
- Gauss-Seidel error: $\|x - x_{exact}\| \approx 0.08$
- Jacobi error: $\|x - x_{exact}\| \approx 0.3$
- Gauss-Seidel converges about 2× faster

8 Practical Implementation & Considerations

8.1 Stopping Criteria

1. **Absolute error:** $\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\| < \epsilon$
2. **Relative error:** $\frac{\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\|}{\|\mathbf{x}^{(k+1)}\|} < \epsilon$
3. **Residual:** $\|\mathbf{b} - A\mathbf{x}^{(k)}\| < \epsilon$

8.2 Acceleration: Successive Over-Relaxation (SOR)

SOR combines old and new values with relaxation parameter ω :

$$x_i^{(k+1)} = (1 - \omega)x_i^{(k)} + \frac{\omega}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij}x_j^{(k)} \right)$$

- $0 < \omega < 1$: Under-relaxation (more stable)
- $\omega = 1$: Standard Gauss-Seidel
- $1 < \omega < 2$: Over-relaxation (faster convergence)

8.3 When to Use Gauss-Seidel

- ✓ **Use when:** System is diagonally dominant or SPD, matrix is sparse, sequential computation is acceptable
- ✗ **Avoid when:** System is ill-conditioned, parallel computation is required, strict diagonal dominance is absent

9 Summary of Key Formulas

Concept	Formula
Gauss-Seidel Iteration	$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij}x_j^{(k)} \right)$
Matrix Decomposition	$A = D + L + U$
Iterative Matrix	$C = -(D + L)^{-1}U$
Convergence Criterion	$\rho(C) < 1$ or $\ C\ < 1$
Frobenius Norm	$\ C\ _F = \sqrt{\sum_{i=1}^n \sum_{j=1}^n c_{ij} ^2}$
Condition Number	$\kappa(A) = \ A\ \ A^{-1}\ $
Diagonal Dominance	$ a_{ii} > \sum_{j \neq i} a_{ij} $ for all i

Table 3: Key formulas for Gauss-Seidel method

10 Practice Problems with Solutions

Step-by-Step Problem

Problem 6: Convergence Analysis For $A = \begin{bmatrix} 5 & 1 & 2 \\ 2 & 6 & 1 \\ 1 & 0 & 4 \end{bmatrix}$:

- Check diagonal dominance
- Normalize to unit diagonal
- Compute iterative matrix C
- Calculate $\|C\|_F$
- Will Gauss-Seidel converge?

✓ Solution

(a) Diagonal dominance check:

- Row 1: $|5| > |1| + |2| = 3$
- Row 2: $|6| > |2| + |1| = 3$
- Row 3: $|4| > |1| + |0| = 1$

Matrix is strictly diagonally dominant.

(b) Normalize (divide each row by diagonal):

$$\tilde{A} = \begin{bmatrix} 1 & 0.2 & 0.4 \\ 0.333 & 1 & 0.1667 \\ 0.25 & 0 & 1 \end{bmatrix}$$

(c) Decompose: $\tilde{A} = I + L + U$

$$L = \begin{bmatrix} 0 & 0 & 0 \\ 0.333 & 0 & 0 \\ 0.25 & 0 & 0 \end{bmatrix}, \quad U = \begin{bmatrix} 0 & 0.2 & 0.4 \\ 0 & 0 & 0.1667 \\ 0 & 0 & 0 \end{bmatrix}$$

$$(I + L)^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ -0.333 & 1 & 0 \\ -0.1667 & 0 & 1 \end{bmatrix}$$

$$C = -(I + L)^{-1}U = \begin{bmatrix} 0 & -0.2 & -0.4 \\ 0 & 0.0667 & -0.0667 \\ 0 & 0.0333 & 0.0667 \end{bmatrix}$$

(d) Frobenius norm:

$$\begin{aligned} \|C\|_F &= \sqrt{(-0.2)^2 + (-0.4)^2 + (0.0667)^2 + (-0.0667)^2 + (0.0333)^2 + (0.0667)^2} \\ &= \sqrt{0.04 + 0.16 + 0.00445 + 0.00445 + 0.00111 + 0.00445} \\ &= \sqrt{0.21446} \approx 0.463 \end{aligned}$$

(e) Since $\|C\|_F = 0.463 < 1$, convergence is guaranteed.

References & Further Reading

1. Golub, G. H., & Van Loan, C. F. (2013). *Matrix Computations*. Johns Hopkins University Press.
2. Saad, Y. (2003). *Iterative Methods for Sparse Linear Systems*. SIAM.
3. Burden, R. L., & Faires, J. D. (2010). *Numerical Analysis*. Cengage Learning.
4. Strang, G. (2019). *Linear Algebra and Learning from Data*. Wellesley-Cambridge Press.